CYBERSECURITY GUIDE BOOK

CHAPTER 4

Web Technologies

Roadmap step 4 of 12

Master the fundamentals first, then specialize.
Every concept is shown from two sides: how an attacker abuses it, and how a defender (SOC analyst) detects and stops it.

Author: ashihh07
For learning and authorized lab use only.

> *You cannot attack or defend what you do not understand. This chapter takes a web page apart piece by piece, so that Chapter 5 (Web Security) is about bugs in things you already know.*

About this chapter

Chapter 1 gave you the network, Chapter 2 the operating system, Chapter 3 the code. Now they come together in the thing most attacks target: the web application. Almost every organisation exposes a website, a login page or an API, and web logs are among the busiest data sources a SOC sees.

The roadmap lists four items:

- **HTML and CSS:** what the browser receives and how it is shown.
- **JavaScript (in context):** how the page behaves and talks to servers.
- **Cookies and sessions:** how a stateless protocol remembers who you are.
- **Web servers (Apache, Nginx):** the software that delivers it all, and keeps the logs.

How each topic is built

- **Concept**: what it is and why it exists.
- **Attacker view**: how it gets abused (for understanding and authorized testing only).
- **Defender view**: how to build it safely and how to spot abuse.
- **Labs**: everything runs on your own VM or inside your own browser.

Contents

#	Section	Roadmap item covered
1	How a web page is delivered	Orientation
2	HTML	HTML, CSS
3	CSS	HTML, CSS
4	JavaScript in the page	JavaScript
5	Cookies, sessions and tokens	Cookies, Sessions
6	Web servers: Nginx and Apache	Web Servers (Apache, Nginx)
7	The whole stack and its logs	Everything together
8	Hands-on labs	Practice for all of the above
9	Quick revision sheet	Tables and commands
10	Self-check quiz and answers	Test yourself

Ground rule for every lab in this book

- Only scan, capture or test systems you own or have written permission to test.
- Run web servers and test pages inside your own VM. Do not expose lab servers to the internet.
- Unauthorized access can be illegal even if you mean no harm.

1. How a web page is delivered

Concept

When you type `https://shop.example.com/login?next=/cart` and press Enter, almost everything from Chapters 1 to 3 happens in order:

```
1. DNS lookup        shop.example.com -> 203.0.113.50                    (Chapter 1)
2. TCP handshake     SYN, SYN-ACK, ACK to port 443                       (Chapter 1)
3. TLS handshake     certificate checked, session key agreed            (Chapter 1)
4. HTTP request      GET /login?next=/cart with Host and Cookie headers  (Chapter 1)
5. Server work       web server finds a file or hands off to application code
6. HTTP response     200 OK + headers + HTML                             (Chapter 1)
7. Browser parses    HTML, then fetches CSS, JavaScript, images, fonts
8. JavaScript runs   may call APIs with fetch and update the page        (Chapter 3)
```

Anatomy of a URL

```
https://shop.example.com:443/login?next=/cart#top
```

Part	In the example	Meaning
Scheme	`https`	How to talk to the server
Host	`shop.example.com`	Where, found through DNS
Port	`443`	Defaults to 443 for HTTPS and 80 for HTTP
Path	`/login`	Which resource
Query	`?next=/cart`	Parameters: `?key=value&key2=value2`
Fragment	`#top`	Stays in the browser, never sent to the server

- The **query** and **path** are attacker-controlled input.
- An **origin** is scheme + host + port. The same-origin policy (Chapter 3) is built on it.

Static vs dynamic, frontend vs backend

Term	Meaning	Example
Static site	Server returns files exactly as stored	A `index.html` portfolio
Dynamic site	Server runs code and builds the page per request	Login, shopping cart, dashboards
Frontend	What runs in the browser: HTML, CSS, JavaScript	The login form
Backend	What runs on the server: application code, database	Checks the password, queries SQL
API	A backend interface that returns data (often JSON) instead of pages	`GET /api/users/42`

> **Rule of the web:** everything the browser holds is under the user's control. They can read it, edit it, replay it, or skip the browser entirely with `curl`. Security checks that only happen in the frontend are decoration.

Your best tool: browser DevTools

Press **F12**. The tabs you will use constantly:

Tab	Shows	Security use
Elements	The live HTML and CSS	See what scripts changed, edit any value
Network	Every request and response, with headers and bodies	Watch cookies, tokens and API calls
Application (Storage in Firefox)	Cookies, localStorage, sessionStorage	Inspect session cookies and their flags
Sources	JavaScript files, breakpoints	Read client-side logic and hidden endpoints
Console	Run JavaScript on the page	Test XSS payloads on your own pages

2. HTML

Concept

HTML (HyperText Markup Language) describes the structure of a page with **elements**: tags, attributes and content.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Lab Login</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Welcome</h1>
  <p>Read the <a href="/help" target="_blank" rel="noopener">help page</a>.</p>

  <form action="/login" method="post">
    <input type="text" name="username" required>
    <input type="password" name="password" required>
    <input type="hidden" name="role" value="user">
    <button type="submit">Log in</button>
  </form>

  <script src="app.js" defer></script>
</body>
</html>
```

Elements you will meet in every assessment

Element	Purpose	Why security people care
`<a href>`	Link	Where users can be sent. Phishing hides the real target
`<form>`	Sends user input to the server	The main attack surface
`<input>`, `<textarea>`, `<select>`	Input fields	Every field is a place to test for injection
`<script>`	Loads or contains JavaScript	Injected scripts are XSS
`<iframe>`	Embeds another page	Clickjacking and untrusted content
`<img>`, `<link>`	Loads images and stylesheets	Can leak that a page was opened (tracking)
`<meta>`	Page information, sometimes redirects or CSP	Policies and `refresh` redirects
`<!-- comment -->`	Developer notes	Visible to everyone

Forms: GET vs POST

	GET	POST
Where the data goes	In the URL: `/search?q=nmap`	In the request body
Appears in	Browser history, server logs, proxy logs, `Referer` header	Request body (not in the URL)

	GET	POST
Use for	Reading and searching	Logging in, changing or creating data
Never use for	Passwords, tokens, personal data	n/a (but still needs HTTPS and checks)

A password sent by GET lands in every access log along the path. That is a finding by itself.

Attacker view

- **View source and comments:** `Ctrl+U` shows the raw HTML. Comments often contain TODOs, old paths, test credentials or API hints.
- **Hidden fields are not secret.** In the form above, `role=user` can be edited to `role=admin` in DevTools, or the request replayed with `curl -d "role=admin"`. If the server trusts it, that is a privilege bug.
- **Client-side validation is bypassable.** `required`, `maxlength` and JavaScript checks vanish when you send the request without the browser.
- **Links with `target="_blank"`** without `rel="noopener"` can let the opened page reach back into the original tab (modern browsers now imply `noopener`, but write it anyway).
- **Cloned pages:** phishing kits copy a login page's HTML and CSS, and only change `action` to the attacker's server.

Defender view

- Treat every field, hidden field, header and cookie as untrusted. **Validate on the server.** Client checks are only for user convenience.
- Never store prices, roles or permissions in hidden fields. Keep them server-side.
- Remove comments and debug text from production pages. Add `autocomplete="off"` only where it helps, and use `type="password"`.
- Frame protection: send `X-Frame-Options: DENY` or CSP `frame-ancestors 'none'` on pages that must not be embedded.
- For a phishing alert, compare the form's `action` host with the page's host. A bank login that posts to another domain is the evidence.

3. CSS

Concept

CSS (Cascading Style Sheets) controls how HTML looks: colours, fonts, layout, spacing. A rule picks elements with a **selector** and sets **properties**.

```
/* element, class, id selectors */
body        { font-family: sans-serif; margin: 0; }
.warning    { color: #c62828; font-weight: bold; }
#login-box  { width: 320px; padding: 16px; border: 1px solid #ccc; }
button:hover { background: #1a237e; color: white; }
```

```
<link rel="stylesheet" href="style.css">     <!-- external file -->
<style> .hidden { display: none; } </style>   <!-- inside the page -->
<p class="warning" style="margin:0">inline style</p>
```

- **Class** (`.name`) is reusable. **ID** (`#name`) is unique per page.
- **The box model:** every element is content, plus padding, plus border, plus margin.
- **Cascade:** when rules collide, the more specific selector wins, and inline styles beat stylesheets.

Why a security book includes CSS

Idea	Meaning
Hidden is not protected	`display:none` only hides something from the eye. The element and its data are still in the page source
Overlays enable clickjacking	An invisible `<iframe>` (`opacity:0`) positioned over a visible button tricks the user into clicking the real page underneath
Visual deception	Fake address bars, look-alike buttons and copied branding make phishing pages convincing
Style injection	If an attacker can inject CSS, they can reshape the page, hide warnings, or in some setups leak data

Attacker view

- Frame a victim site invisibly and place a decoy button above its "Delete account" button (**clickjacking**).
- Build a pixel-perfect copy of a login page using the original's stylesheet and logo.

Defender view

- Block framing with `X-Frame-Options` or `frame-ancestors` .
- Use a Content-Security-Policy that limits where styles and scripts can load from.
- Do not put sensitive data in the page "but hidden". If the user should not see it, do not send it.
- Brand-new domains using your logo and CSS are a detection signal for phishing monitoring.

4. JavaScript in the page

Concept

Chapter 3 taught the language. Here is how it lives inside a page.

```
<script src="app.js" defer></script>       <!-- external file, runs after the HTML is parsed
-->
<script> console.log("inline script"); </script>   <!-- inline code -->
<button onclick="doThing()">Click</button>   <!-- inline event handler -->
```

Events connect user actions to code: `click`, `submit`, `input`, `keydown`, `load`. **AJAX** (`fetch`) lets a page request data in the background and update itself without reloading. Modern sites are mostly one HTML shell plus JavaScript that calls APIs:

```
document.querySelector("form").addEventListener("submit", async (e) => {
  e.preventDefault();                                  // stop normal form submit
  const body = { username: e.target.username.value };
  const r = await fetch("/api/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body),
  });
  console.log(await r.json());                         // the API's JSON answer (Chapter 3)
});
```

Third-party scripts and supply chain

Most pages load scripts from other sites: analytics, chat widgets, ad networks, CDNs. **Each one runs with full rights inside your page.** If that provider is compromised, so is every site that includes it.

```
<script src="https://cdn.example.com/lib.js"
        integrity="sha384-BASE64HASHGOESHERE"
        crossorigin="anonymous"></script>
```

Subresource Integrity (`integrity=`) makes the browser refuse the file if its hash does not match.

Reading client-side code

- **Minified** code has no spaces and short names to save bytes. **Obfuscated** code is scrambled on purpose. Both can be prettified in DevTools (the `{}` button in Sources).
- **Source maps** (`.js.map`) rebuild the original code and sometimes expose internal paths and comments.
- Search JavaScript for `/api/`, `token`, `apiKey`, `admin`, `debug`, `fetch(` to find endpoints the interface never shows.

Attacker view

- Read the JavaScript to list every API endpoint, then call them directly without the interface.
- Find secrets shipped to the browser: API keys, tokens, internal URLs, test credentials.
- Poison a third-party script to run code on every site that includes it.
- Use `fetch` from injected script (XSS) to act as the victim.

Defender view

- **Never put secrets in front-end code.** Anything sent to the browser is public.
- Use **SRI** for third-party scripts, pin versions, and keep the list of included domains short.
- Send a **Content-Security-Policy** such as `script-src 'self'` so injected and unexpected scripts do not run.
- Make every API enforce authentication and authorization on the server. Hiding a button is not access control.
- Do not publish source maps for production unless you accept that code being readable.

5. Cookies, sessions and tokens

Concept

HTTP is **stateless**: the server forgets you after each response. To remember a logged-in user, the server gives the browser a small value, and the browser sends it back with every request.

```
1. POST /login  (username, password)                -> browser to server
2. Server checks credentials, creates session #a94f..  stored on the server
3. HTTP/1.1 200 OK
   Set-Cookie: session=a94f...; Secure; HttpOnly; SameSite=Lax; Path=/    <- server to
browser
4. GET /account
   Cookie: session=a94f...                          <- browser sends it automatically every
time
5. Server looks up a94f..., finds "alice", returns her page
6. POST /logout -> server deletes session #a94f..    (the cookie must stop working)
```

Whoever holds the session cookie is the user. The server cannot tell the difference between Alice's browser and a thief with a copy of her cookie.

Cookie attributes

Attribute	What it does	Why it matters
Secure	Cookie is sent only over HTTPS	Stops theft on plain HTTP
HttpOnly	Hidden from JavaScript (`document.cookie`)	Limits damage from XSS
SameSite	`Strict` , `Lax` or `None` : limits sending on cross-site requests	Main defence against CSRF (Chapter 5). `None` requires `Secure`
Domain / Path	Which hosts and paths receive it	Too broad a Domain shares it with every subdomain
Max-Age / Expires	Lifetime. Without them it is a session cookie, gone when the browser closes	Shorter life means less time to abuse a stolen cookie
`__Host-` prefix	Name rule: must be Secure, Path=/, with no Domain	Stops other subdomains from overwriting it

What a good session ID looks like

- **Random and long:** at least 128 bits from a cryptographic generator (`secrets.token_urlsafe(32)` in Python). Never a counter, a username or a timestamp.
- **New after login:** issue a fresh ID when the user authenticates (stops **session fixation**, where an attacker plants a known ID first).
- **Destroyed on logout** and after idle and absolute timeouts.
- **Stored server-side**, so the server can revoke it.

Tokens: JWT

Many APIs use **JSON Web Tokens** instead of server-side sessions. A JWT is three base64url parts separated by dots:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 . eyJzdWIiOiJhbGljZSIsInJvbGUiOiJ1c2VyIn0 .
dGhpcyBpcyB0aGUgc2lnbg
            header                                    payload
signature
{"alg":"HS256","typ":"JWT"}              {"sub":"alice","role":"user"}
```

> A JWT is **signed, not encrypted.** Anyone can decode and read the payload without any key. The signature only proves it was not changed. Never put secrets in it, and **always verify the signature on the server**.

	Server-side session cookie	JWT token
State	Stored on the server	Self-contained, held by the client
Revoking	Delete the session: instant	Hard until it expires, unless you keep a block list
Usual storage	Cookie (`HttpOnly`)	Cookie or `localStorage`
Typical mistakes	Predictable IDs, no expiry, no regeneration	Signature not checked, `alg: none` accepted, weak secret, long lifetime

Where to keep a token in the browser

Storage	Readable by JavaScript?	Sent automatically?	Main risk
HttpOnly cookie	No	Yes	CSRF (use `SameSite`)
localStorage / sessionStorage	Yes	No	Stolen by any XSS

> **Attacker view**
>
> - **Cookie theft:** by XSS (`document.cookie` , when `HttpOnly` is missing), by sniffing plain HTTP (when `Secure` is missing), or by malware on the user's device.
> - **Session fixation:** make the victim use a session ID the attacker already knows.
> - **Predictable or unrevoked sessions:** guess IDs, or reuse a stolen one long after the user logged out.
> - **JWT tampering:** change `"role":"user"` to `admin` and hope the server skips signature checks, or accepts `alg: none` .
> - **CSRF:** a malicious page makes the victim's browser send a request with their cookie attached.

Defender view

- Set `Secure; HttpOnly; SameSite=Lax` (or `Strict`) on every session cookie. Prefer the `__Host-` prefix.
- Use long random IDs, regenerate on login, expire on idle and absolute time, invalidate on logout and on password change.
- Verify JWT signatures with a fixed, strong algorithm. Keep lifetimes short and use refresh tokens.
- **SOC signal:** one session ID used from two countries or two user agents, or a login followed at once by sensitive actions, suggests a stolen session.
- Log session creation, logout and failures. A spike of 401/403 responses points to guessing or replay.

6. Web servers: Nginx and Apache

Concept

A **web server** listens on ports 80 and 443, receives HTTP requests, and either returns a file from disk (static) or passes the request to application code (dynamic). It writes the **access log** and **error log** that you will analyse for years.

	Nginx	Apache HTTP Server
Design	Event-driven, lightweight, very fast with many connections	Process/thread based, very flexible modules
Common role	Front door, **reverse proxy**, load balancer, static files	Classic hosting, PHP applications, per-directory rules
Per-folder config	No (central config only)	Yes (`.htaccess`)
Debian config	`/etc/nginx/nginx.conf`, `sites-available/`, `sites-enabled/`	`/etc/apache2/apache2.conf`, `sites-available/`, `mods-enabled/`
Web root	`/var/www/html`	`/var/www/html`
Logs	`/var/log/nginx/access.log`, `error.log`	`/var/log/apache2/access.log`, `error.log`
Runs as	`www-data` (Debian), `nginx` (Red Hat)	`www-data` (Debian), `apache` (Red Hat)
Test config	`sudo nginx -t`	`sudo apache2ctl configtest`
Apply change	`sudo systemctl reload nginx`	`sudo systemctl reload apache2`

Install and check

```
sudo apt install nginx                  # or: sudo apt install apache2
systemctl status nginx
curl -I http://127.0.0.1                # see the Server header and status
ss -tulpn | grep ':80'                  # who is listening (Chapter 1 and 2 commands)
```

Nginx and Apache both want port 80, so run only one at a time (`sudo systemctl stop nginx` before you start Apache).

A minimal site (Nginx server block)

```
server {
    listen 80;
    server_name lab.local;
    root /var/www/lab;
    index index.html;

    server_tokens off;                                  # hide the version number
    autoindex off;                                      # no directory listings
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;

    location / {
        try_files $uri $uri/ =404;                      # serve file, or 404
    }
}
```

Save it as `/etc/nginx/sites-available/lab`, link it with `sudo ln -s /etc/nginx/sites-available/lab /etc/nginx/sites-enabled/`, then `sudo nginx -t && sudo systemctl reload nginx`. Add `127.0.0.1 lab.local` to `/etc/hosts` (the local DNS override from Chapter 1) to reach it by name.

The same in Apache

```
<VirtualHost *:80>
    ServerName lab.local
    DocumentRoot /var/www/lab

    <Directory /var/www/lab>
        Options -Indexes                # no directory listings
        AllowOverride None              # ignore .htaccess in this tree
        Require all granted
    </Directory>

    Header always set X-Content-Type-Options "nosniff"
    Header always set X-Frame-Options "DENY"
</VirtualHost>
```

```
sudo a2enmod headers                    # the Header lines need this module
sudo a2ensite lab.conf
sudo apache2ctl configtest && sudo systemctl reload apache2
```

On Debian, also set `ServerTokens Prod` and `ServerSignature Off` in `/etc/apache2/conf-available/security.conf` so the version number is not shown.

Virtual hosts and reverse proxies

- A **virtual host** (Apache) or **server block** (Nginx) lets one server host many sites, picked by the `Host` header.

- A **reverse proxy** accepts the request and forwards it to an application server behind it. It terminates TLS, adds headers, caches, and applies rate limits. Attackers see the proxy, not the application.
- Behind a proxy, the application sees the proxy's IP unless the real client IP is passed in `X-Forwarded-For`. In logs that is the same "which internal host was it?" problem as NAT in Chapter 1.

Reading an access log line

```
203.0.113.5 - - [22/Oct/2026:10:15:03 +0000] "GET /login?next=/cart HTTP/1.1" 401 512 "-"
"curl/8.4.0"
```

Field	`awk` position	Value here
Client IP	`$1`	`203.0.113.5`
Time	`$4`	`[22/Oct/2026:10:15:03`
Method	`$6`	`"GET`
Path	`$7`	`/login?next=/cart`
Status code	`$9`	`401`
Bytes sent	`$10`	`512`
User agent	`$6` when splitting on quotes (`-F'"'`)	`curl/8.4.0`

```
awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -nr          # status code
counts
awk '$9==404 {print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head   # who
causes 404s
awk '$9==401 || $9==403 {print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -nr |
head
grep -E "\.\./|%2e%2e|/etc/passwd" /var/log/nginx/access.log                    # traversal
attempts
awk -F'"' '{print $6}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head   # user
agents
```

Attacker view

- **Banners:** `Server: Apache/2.4.29 (Ubuntu)` tells which public exploits to try.
- **Directory listing** exposes file names, backups (`.bak` , `.old` , `.zip` , `.sql`) and configuration.
- **Default and leftover files:** default welcome pages, `phpinfo.php` , `/server-status` , admin panels, `/.git/config` (a Git repository inside the web root).
- **Path traversal:** `../../etc/passwd` in a URL, in plain or encoded form (`%2e%2e%2f`), to escape the web root.
- **Web shells:** a malicious script uploaded into the web root and then called by URL.
- **Reading the unpatched server:** old Nginx and Apache versions have known CVEs.

Defender view: hardening checklist

- **Patch** the server and every module. Hide version banners (`server_tokens off` , `ServerTokens Prod`).
- **No directory listing** (`autoindex off` , `Options -Indexes`). Remove default pages, test files and backups from the web root.
- **Least privilege:** the server runs as `www-data` , which owns nothing important. The web root should not be writable by the web user unless an upload folder needs it.
- **Block sensitive paths:** deny `/.git` , `/.env` , `*.bak` , `*.sql` .
- **Send security headers** and use HTTPS only (Chapter 1), with HSTS.
- **Limit methods** to what the site needs, set request size limits, and add **rate limiting** (`limit_req` in Nginx).
- **Detect web shells:** `find /var/www -type f -name "*.php" -mtime -1` shows recently created or changed scripts.
- **Forward logs** to a SIEM. Alert on bursts of 404s, 401/403s, odd user agents, and traversal strings.

7. The whole stack and its logs

Real sites have layers. Each layer is a place to attack and a place to log:

```
Browser  ->  CDN / WAF  ->  Reverse proxy (Nginx)  ->  Application server  ->  Database
 user       filters bad     TLS, caching, rate        business logic,         stores data
 cookies    traffic         limits, access log        sessions, API           (SQL, Chapter
5)
```

Layer	Log you read	Question it answers
CDN / WAF	Blocked and allowed requests, rule matches	Was this attack already stopped at the edge?
Reverse proxy	`access.log` , `error.log`	Who asked for what, what status, how many bytes?
Application	Application log, authentication events	Which user, which action, did the login succeed?
Database	Query and error log	What was actually queried?
Host	`auth.log` , `syslog` , process list (Chapter 2)	Did anyone get onto the server itself?

A single incident usually leaves a trail in **several** of these. A SQL injection attempt shows as a strange URL in the proxy log, an error in the application log, and a failed query in the database log. Correlating them is the job.

8. Hands-on labs

Do these in your VM or your own browser. Save a short write-up of each (goal, commands, screenshot, what you learned).

Lab 1: DevTools tour

Open any site you use. In DevTools:

- **Network:** reload, click the first request, read request and response headers. Find `Set-Cookie`, `Content-Type`, `Server`.
- **Application:** list the cookies and read their flags (Secure, HttpOnly, SameSite).
- **Sources:** find one `.js` file and search it for `fetch(` or `/api/`.
- **Elements:** change a heading's text. Reload and watch it vanish.

Goal: explain which cookies on that site are readable by JavaScript and which are not.

Lab 2: Build a tiny site

```
mkdir -p ~/weblab && cd ~/weblab
cat > index.html <<'EOF'
<!doctype html>
<html><head><meta charset="utf-8"><title>Lab</title>
<link rel="stylesheet" href="style.css"></head>
<body>
<h1>Search</h1>
<form action="results.html" method="get">
  <input name="q" placeholder="search"><input type="hidden" name="role" value="user">
  <button>Go</button>
</form>
<script src="app.js" defer></script>
</body></html>
EOF
echo 'body{font-family:sans-serif;margin:2rem} h1{color:#1a237e}' > style.css
echo 'console.log("page loaded");' > app.js
echo '<h1>Results</h1><p>Check the URL.</p>' > results.html
python3 -m http.server 8080
```

Open `http://127.0.0.1:8080`, search for `nmap`, and read the address bar.

Goal: see the query string (`?q=nmap&role=user`) in the URL. Change `role` to `admin` in DevTools before submitting and explain what that proves about hidden fields.

Lab 3: Issue and replay a session cookie

Save as `cookie_server.py` and run `python3 cookie_server.py`:

```
from http.server import BaseHTTPRequestHandler, HTTPServer
import secrets

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        cookie = self.headers.get("Cookie", "")
        if "session=" not in cookie:
            sid = secrets.token_urlsafe(32)
            body = f"New session issued: {sid}\n"
            set_cookie = f"session={sid}; HttpOnly; SameSite=Lax; Path=/"
        else:
            body = f"Welcome back. Your cookie: {cookie}\n"
            set_cookie = None
        data = body.encode()
        self.send_response(200)
        if set_cookie:
            self.send_header("Set-Cookie", set_cookie)
        self.send_header("Content-Type", "text/plain")
        self.send_header("Content-Length", str(len(data)))
        self.end_headers()
        self.wfile.write(data)

HTTPServer(("127.0.0.1", 8000), Handler).serve_forever()
```

```
curl -i http://127.0.0.1:8000/                    # no cookie: server issues one
curl -i -c jar.txt http://127.0.0.1:8000/         # save the cookie
curl -i -b jar.txt http://127.0.0.1:8000/         # replay it: "Welcome back"
```

Goal: show that anyone who copies the cookie value gets the same identity, without any password. This lab omits `Secure` because it runs on plain local HTTP; a real site must set it.

Lab 4: Decode a JWT without any key

```
import base64, json

def b64u(b):  return base64.urlsafe_b64encode(b).rstrip(b"=").decode()
def dec(s):   return json.loads(base64.urlsafe_b64decode(s + "=" * (-len(s) % 4)))

header  = b64u(json.dumps({"alg": "HS256", "typ": "JWT"}).encode())
payload = b64u(json.dumps({"sub": "alice", "role": "user"}).encode())
token = f"{header}.{payload}.fake-signature"
print(token)
print(dec(token.split(".")[1]))          # readable with no secret
```

Goal: explain why a server that does not verify the signature would accept a token where you changed `role` to `admin`.

Lab 5: Serve a site with Nginx

```
sudo apt install nginx
sudo mkdir -p /var/www/lab && echo "<h1>lab</h1>" | sudo tee /var/www/lab/index.html
curl -I http://127.0.0.1                      # note the Server header with version
```

Create the server block from section 6, enable it, add `lab.local` to `/etc/hosts`, then run `sudo nginx -t && sudo systemctl reload nginx` and `curl -I http://lab.local`.

Goal: the `Server` header no longer shows a version, and your two security headers appear.

Lab 6: Directory listing, on then off

```
sudo mkdir /var/www/lab/files && echo "secret-ish" | sudo tee /var/www/lab/files/notes.txt
```

Add `autoindex on;` inside `location /` in your server block, reload, and open `http://lab.local/files/`. Then set `autoindex off;`, reload, and request it again.

Goal: see the listing, then the `403`. Write one sentence on what an attacker could learn from the listing.

Lab 7: Read your own access log

```
for p in / /missing /admin /.git/config "/../../etc/passwd" /login; do curl --path-as-is -s -o /dev/null "http://lab.local$p"; done
sudo tail -n 10 /var/log/nginx/access.log
sudo awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -nr
sudo awk '$9==404 {print $1, $7}' /var/log/nginx/access.log
```

`--path-as-is` stops curl from tidying the `../` out of the path before sending, so the traversal string really reaches the server.

Goal: find your own probing in the log and write the one-line detection rule that would flag it (for example "more than 5 `404` from one IP in a minute").

Lab 8: Same site on Apache

```
sudo systemctl stop nginx
sudo apt install apache2
```

Rebuild the virtual host from section 6, enable `headers`, set `ServerTokens Prod`, reload, and compare `curl -I` output with Nginx.

Goal: a short table of "Nginx setting versus Apache setting" for banner hiding, directory listing and headers, written in your own words.

> **Next step on TryHackMe:** work through the web fundamentals rooms in the Pre Security path (how websites work, HTTP in detail, DevTools), then write them up the same way.

9. Quick revision sheet

The web in tables

Topic	Remember
URL	scheme, host, port, path, query, fragment. The fragment never reaches the server
HTML	Structure. Forms are the attack surface. Hidden fields and comments are public
CSS	Looks only. `display:none` is not protection. Enables clickjacking and fake pages
JavaScript	Runs in the browser. Never put secrets in it. Third-party scripts run with full rights
Cookies	`Secure`, `HttpOnly`, `SameSite`, short life, `__Host-` prefix
Sessions	Long random ID, new on login, destroyed on logout, stored server-side
JWT	Signed, not encrypted. Verify the signature. Keep it short-lived
Web servers	Hide banners, no directory listing, least privilege, security headers, rate limits, send logs to SIEM

Commands

Task	Command
See headers	`curl -I http://host`
Full exchange	`curl -v http://host`
Save and replay cookies	`curl -c jar.txt URL`, then `curl -b jar.txt URL`
Test Nginx config	`sudo nginx -t`
Test Apache config	`sudo apache2ctl configtest`
Reload without dropping connections	`sudo systemctl reload nginx` (or `apache2`)
Status code counts	`awk '{print $9}' access.log | sort | uniq -c | sort -nr`
Top clients	`awk '{print $1}' access.log | sort | uniq -c | sort -nr | head`
Recently changed web files	`find /var/www -type f -mtime -1`

One-line summaries

- **Everything the browser holds belongs to the user.** Check on the server.
- **GET shows in URLs and logs.** Use POST for anything private.
- **A session cookie is the user.** Protect it with `Secure; HttpOnly; SameSite`.
- **A JWT is readable by everyone.** Only the signature protects it.
- **A web server's job is also to leave evidence.** Read the access log.

10. Self-check quiz

Answer without looking back, then open the key.

1. What part of a URL is never sent to the server?
2. Why should a password never be sent with a GET request?
3. Why is `<input type="hidden" name="role" value="user">` not a security control?
4. What does the `HttpOnly` cookie flag prevent?
5. What does `SameSite` mainly defend against?
6. Is a JWT encrypted? What does the signature actually prove?
7. Name two differences between Nginx and Apache.
8. Which two settings hide the web server version and directory listings in Nginx?
9. In a combined access log line, which `awk` field is the status code?
10. A web server process spawns `/bin/sh`, and a new `.php` file appeared in `/var/www` an hour ago. What do you suspect?

> **Show answer key**
>
> 1. The fragment (the part after `#`).
> 2. The URL ends up in browser history, server and proxy logs, and `Referer` headers, exposing the password in many places.
> 3. The user controls the browser and can edit the value or replay the request without the page. The server must decide the role itself.
> 4. JavaScript reading the cookie (`document.cookie`), which limits what an XSS bug can steal.
> 5. Cross-site request forgery (CSRF): it stops the browser from sending the cookie on most cross-site requests.
> 6. No. It is only base64url encoded, so anyone can read it. The signature proves the content was not changed by someone without the key.
> 7. Any two of: event-driven vs process/thread based; Nginx has no `.htaccess`; Nginx is common as a reverse proxy; Apache has dynamic modules and per-directory config.
> 8. `server_tokens off;` and `autoindex off;`.
> 9. `$9`.
> 10. A web shell: an attacker uploaded a script and used it to run commands. Isolate the host, preserve logs and the file, and check the access log for requests to that file.

What comes next

Chapter 5: Web Security. You now know how a page, a cookie and a web server work. Next you attack and defend them: SQL injection, cross-site scripting, CSRF, authentication and authorization flaws, and the OWASP Top 10.